VULNERABILITY ASSESSMENT REPORT

OS Command Injection via Filtered Network Diagnostic (Ping) Utility

Target Application: Yavuzlar Vulnerable Web Lab ("Ping" / Network Diagnostic Module)

by: Reinaldy Thendean

Date: July 2026

Classification: Portfolio / Educational Documentation

Disclaimer: This assessment was performed against "Yavuzlar", a deliberately vulnerable web application built for penetration testing training and self-study purposes. All testing was conducted in an isolated lab environment with explicit permission to attack; no real users, production systems, or real infrastructure were involved. This document is published solely to demonstrate methodology and technical understanding for portfolio purposes.

Report Information

Report Title	OS Command Injection on the Ping / Network Diagnostic Feature via Input Filter Bypass
Target	Yavuzlar Vulnerable Web Application (training vulnerable lab)
Tester	Reinaldy Thendean (Undergraduate Student, Web Application Security)
Testing Type	Manual black-box web application penetration testing
Tools Used	Web Browser, manual payload crafting and command-injection filter fuzzing
Vulnerability Class	OS Command Injection (Blocklist Filter Bypass)
CWE Reference	CWE-78 (OS Command Injection), CWE-693 (Protection Mechanism Failure), CWE-184 (Incomplete Blacklist)
CVSS v3.1 Base Score	9.8 (Critical)
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

1. Executive Summary

During a manual security assessment of the network diagnostic ("Ping") feature of the Yavuzlar training lab, an OS Command Injection vulnerability was identified. The feature accepts a user-supplied IP address / hostname and passes it to an underlying operating-system shell command without adequate sanitization.

The application attempted to defend this feature with a blocklist-style input filter that blocks a limited set of command-separator characters (such as ; and &) and a limited set of literal command keywords (such as "whoami"). Through systematic fuzzing and variable isolation, it was determined that this filter could be bypassed using two independent techniques: (1) case-modification of the blocked keyword, which revealed that the | and || operators were never actually blocked, and (2) string obfuscation of the keyword using backticks (`) and an empty command-substitution sub-shell ($()), which defeated the literal-keyword blocklist while still allowing the Linux shell to reconstruct and execute the intended command.

This resulted in confirmed arbitrary OS command execution in the context of the web server process (running as the www-data user), demonstrated by successfully executing the whoami command through the bypass and observing its output rendered back to the tester.

This finding was rated Critical severity (CVSS v3.1 base score 9.8) because it allows an unauthenticated or low-privileged attacker to execute arbitrary operating-system commands on the server, directly compromising confidentiality, integrity, and availability of the underlying host.

2. Scope

- Application: Yavuzlar vulnerable web application (intentionally vulnerable lab designed for pentest training).
- Module in scope: network diagnostic / "Ping" utility feature, including its server-side input filter.
- Test type: Manual dynamic testing (DAST) with direct input fuzzing; no automated scanners were used.
- Testing was performed by submitting crafted IP-address input values directly into the ping feature and observing the rendered output.

3. Methodology

The assessment followed a standard manual black-box testing approach, using an incremental variable-isolation strategy to characterize the input filter before attempting to bypass it:

1. Confirm baseline functionality of the ping feature with a legitimate, unmodified input.

2. Attempt to chain a second command onto the ping input using common Linux command-separator operators, to determine whether command chaining is possible at all.

3. When chained commands returned an error, isolate whether the error was caused by the separator operator itself or by the literal command keyword, by modifying the keyword's letter case while keeping the operator unchanged.

4. Once the safe operator(s) were identified, systematically test additional special characters and encoding schemes to map the full boundary of the filter.

5. Use the identified unfiltered characters to construct an obfuscated version of the blocked keyword that a literal-string filter would not recognize, while remaining valid shell syntax.

6. Validate successful command execution by observing the output of the injected command.

4. Vulnerability Analysis & Proof of Concept

4.1 Case 1: Command Injection Without a Filter

The first test was performed against a baseline instance of the ping feature with no input filter in place.

Step 1 — Submit a standard ping request: Ping 8.8.8.8.



Figure 1. Normal ping request to 8.8.8.8 executing successfully.

The result was normal, and the ping executed successfully.

Step 2 — Attempt to chain a second command using the pipe operator: 8.8.8.8|whoami.

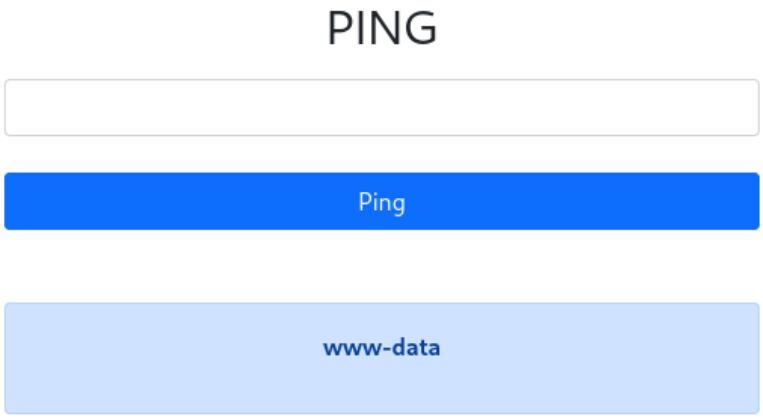


Figure 2. Injected "|whoami" command executing successfully, returning "www-data".

The result showed the output www-data, confirming that the ping input field passes user input directly to an OS-level shell without sanitization, and that arbitrary commands could be chained and executed.

4.2 Case 2: Command Injection With a Filter in Place

A second, filtered instance of the ping feature was then tested to determine whether the same technique would work, and if not, to characterize and bypass whatever protection had been added.

4.2.1 Initial Testing of Command Separators (Initial Fuzzing)

Testing began by attempting to chain the main command (ping) with an additional command (whoami) using various standard Linux separator characters (operators):

```
8.8.8.8;whoami → Error
8.8.8.8&&whoami → Error
8.8.8.8||whoami → Error
8.8.8.8|whoami → Error
```

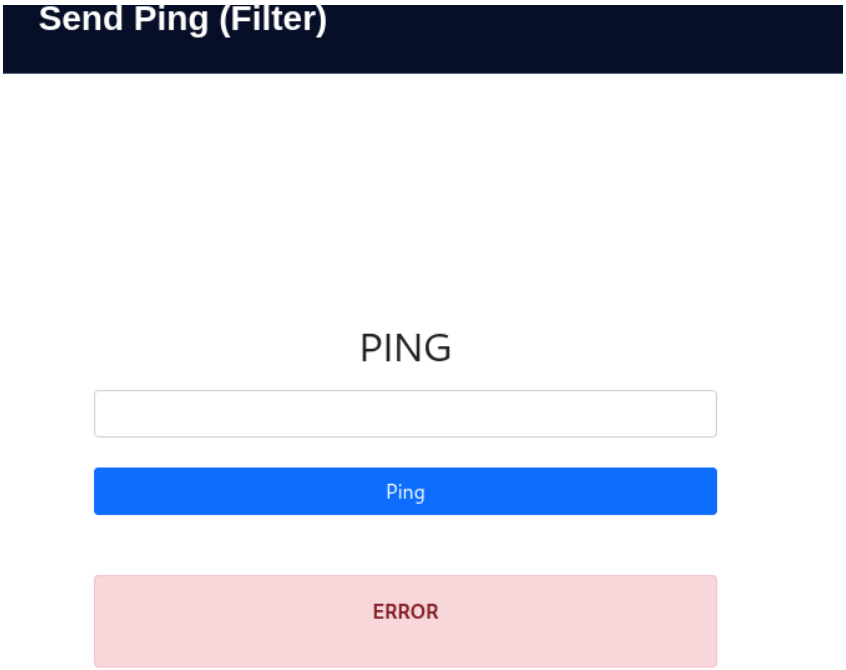


Figure 3. All initial separator/keyword combinations returning a security error.

Stage 1 Analysis:

All combinations resulted in a security error. At this point, the system was in a "blind" state: it could not yet be concluded whether the error was triggered because all operators were blocked, or because the keyword "whoami" itself was blocked by the application.

4.2.2 Isolating the Variable Using a Case-Modification Technique

To break the deadlock and determine which variable actually triggered the error — the operator or the keyword — the tester modified the text to WhOaMi and lS:

```
8.8.8.8;WhOaMi → Error
8.8.8.8&&WhOaMi → Error
8.8.8.8||WhOaMi → Ping result returned (no error)
```

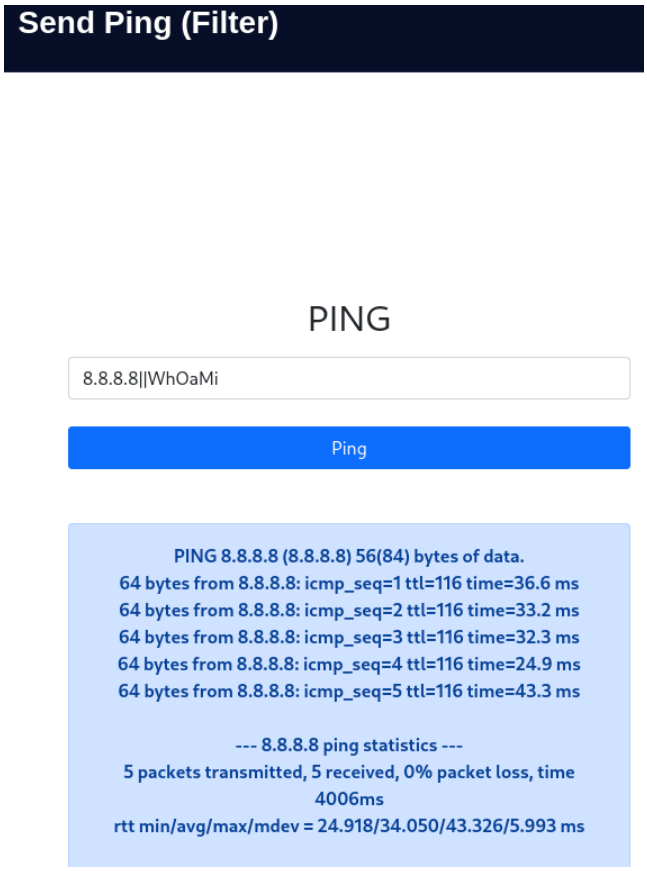


Figure 4. The "||WhOaMi" payload returning a ping result with no security error.

`8.8.8.8|WhOaMi → No error, but no output`

`8.8.8.8|lS-La → No error, but no output`

Stage 2 Analysis (Key Finding):

When the keyword was changed to WhOaMi, the ; and && operators still produced an error. This conclusively proves that the ; and & characters are indeed on the blocked-operator list.

In contrast, the || and | operators no longer triggered a security error. This proves two new facts: the || and | operators are actually safe and not blocked by the application, and the errors seen in Stage 1 (||whoami and |whoami) occurred purely because the keyword "whoami" itself had been detected and blocked by the application filter from the start.

Note: Why was there no output for |WhOaMi? Because the Linux operating system is case-sensitive, so an upper-case command is not recognized by the system (command not found).

4.2.3 Testing the Boundaries of the Special-Character Filter

After determining that the | operator did not trigger a security error, the tester tried further input variations to map out which special characters were disallowed by the system:

`8.8.8.8|lS${IFS}-La → No error, no output (the $ and {} characters appear to pass through the application's filter).`

`8.8.8.8|cat%20/et%5C%0Ac/pa%5C%0Asswd → Error (strong indication that the % character or URL-encoding mechanism is blocked by the application).`

```
8.8.8.8|who$@mi → Error (strong indication that a special character such as @ is
blocked by the application).
```

Filter-Rule Conclusion:

Through this series of tests, the tester was able to map the application's filtering boundaries. The system applies two layers of defense: it blocks certain special characters (such as ;, &, @, %) and it blocks specific command keywords (such as "whoami").

Notes: Screenshots for these boundary tests are omitted as results were consistent errors with no new information beyond confirming the blocked character.

4.2.4 Executing the Bypass Experiment (The Breakthrough)

Based on the finding in the previous stage that the $ and {} characters did not trigger a security error, the tester looked for a technique to split the blocked keyword "whoami" without using any blocked characters. The technique chosen was string obfuscation using the backtick character (`) and an empty sub-shell ($()). The goal was to disguise the word so it would evade the application's filter check, while still allowing the Linux shell to reassemble it during execution in the background.

```
8.8.8.8|w``hoam``i → Bypass successful (whoami output appeared)
```

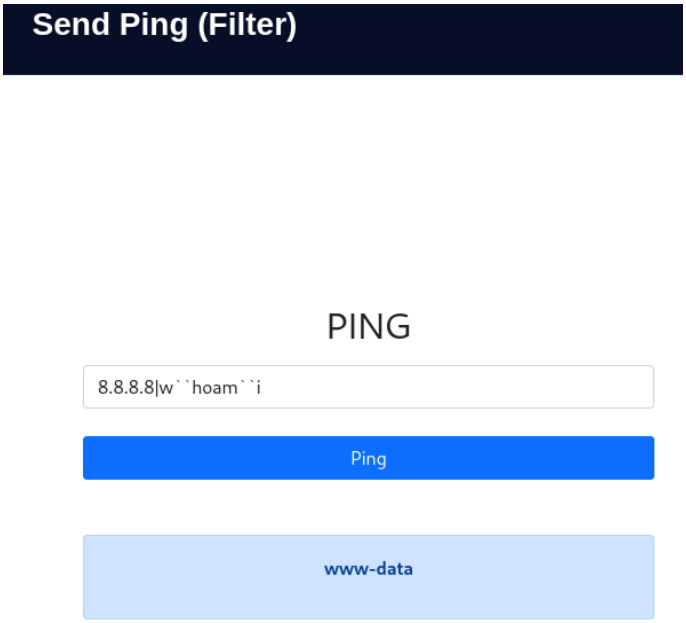


Figure 5. The backtick-obfuscated payload "w``hoam``i" bypassing the filter and returning the whoami output.

```
8.8.8.8|wh$()oami → Bypass successful (whoami output appeared)
```

Final Success Analysis:

The input wh$()oami successfully deceived the application's text-scanning function, because the string was not read literally as the word "whoami". When passed to the backend server, the Linux shell evaluated $() as an empty string, reassembling the separated letters into the valid whoami command, and successfully displayed the server's username on screen.

As a final validation, the input 8.8.8.8|Wh$()OamI was tried, and the result was no error but no output. This reinforces the evidence that the application filter was successfully bypassed (since it did not trigger an error), but failed to display readable text due to the Linux operating system's case-sensitivity toward the WhOamI command.

Together, these two working payloads confirm a fully reproducible OS Command Injection filter bypass, resulting in arbitrary command execution in the context of the web server process.

5. Root Cause Analysis

The underlying weaknesses that make this vulnerability possible are:

1. The ping feature passes user-supplied input directly into an operating-system shell command (e.g., via a function such as exec/system/shell_exec) without proper sanitization, which is the root cause of the OS Command Injection (CWE-78).

2. The application relies on a blocklist-style filter that blocks a limited set of separator characters (;, &, @, %) and a limited set of literal command keywords (e.g., "whoami"), rather than validating input against a strict allow-list (such as a valid IP-address format) or avoiding shell invocation entirely.

3. Blocklist filters that match keywords as literal strings are inherently bypassable: an attacker can split or obfuscate the blocked keyword (e.g., using backticks or an empty $() sub-shell) so that the filter never sees the literal blocked word, while the underlying shell still reconstructs and executes it correctly at runtime.

4. The filter does not block all shell metacharacters — characters such as $, {, and } were not filtered — and these alone are sufficient to defeat a keyword-based blocklist through command substitution and string reassembly.

5. The presence of a working operator (| and ||) together with a shell context that honors command substitution ($()) and backtick-based string splitting confirms that user input reaches a genuine, unrestricted shell interpreter rather than a sanitized or sandboxed execution context.

6. CVSS v3.1 Severity Scoring

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H Base Score: 9.8 (Critical)

Metric	Value	Justification
Attack Vector (AV)	Network (N)	The vulnerable feature is reachable over the web application via HTTP requests.
Attack Complexity (AC)	Low (L)	No special conditions are required beyond crafting an obfuscated payload string; the technique is reliably reproducible.
Privileges Required (PR)	None (N)	The ping/network-diagnostic feature was reachable without requiring elevated application privileges.
User Interaction (UI)	None (N)	No action from another user or victim is required.
Scope (S)	Unchanged (U)	The impact is confined to the vulnerable component/application server itself.
Confidentiality (C)	High (H)	Arbitrary command execution allows an attacker to read sensitive files and data accessible to the web server process.
Integrity (I)	High (H)	Arbitrary command execution allows an attacker to modify files, application data, or configuration on the server.
Availability (A)	High (H)	Arbitrary command execution allows an attacker to disrupt or disable the affected service or host.

Note on impact rating: although this proof of concept only demonstrated execution of the whoami command, the underlying flaw grants execution of arbitrary OS commands in the context of the web server (www-data) user. This is scored C:H/I:H/A:H because the same bypass technique can be used to execute any command reachable by that user, not merely the one demonstrated.

7. Business Impact

- Full compromise of the affected host is possible: an attacker can execute arbitrary operating-system commands with the privileges of the web server process (www-data).
- Sensitive data exposure: configuration files, source code, credentials, and other data accessible to the web server process could be read and exfiltrated.
- Potential for lateral movement: a foothold gained through command execution can be leveraged to pivot further into the internal network or escalate privileges on the host.
- Service disruption: arbitrary command execution could be used to degrade or take down the affected application or host.
- Reputational and compliance risk if this pattern exists in a production application, since OS Command Injection is consistently ranked among the most severe web application vulnerability classes.

8. Remediation Recommendations

1. Avoid invoking OS shell commands with user-supplied input entirely; use safe, language-native networking APIs (e.g., a native ICMP/ping library) instead of shelling out to the system ping binary.

2. If shelling out is unavoidable, use parameterized/argument-array execution APIs (which pass arguments directly to the process without an intermediate shell) instead of concatenating user input into a shell command string.

3. Validate input using a strict allow-list — for example, a regular expression that only accepts a syntactically valid IPv4/IPv6 address or hostname — and reject anything that does not match, rather than trying to block a list of disallowed characters or keywords.

4. Do not rely on blocklists that match literal command keywords; such filters can be defeated through case manipulation, string splitting, encoding, or shell command-substitution syntax such as $().

5. Ensure all shell metacharacters relevant to the target shell (including $, {, }, backticks, and command-substitution syntax) are considered when designing input filters, not only common separators such as ; and &.

6. Apply the principle of least privilege to the web server process so that, even if command execution is achieved, the resulting impact on the host is minimized.

7. Add server-side logging, monitoring, and alerting for anomalous ping input patterns (e.g., inputs containing shell metacharacters or repeated filter-triggering attempts).

8. Include this specific filter-bypass technique (case modification plus backtick/$() obfuscation) in regression and security test suites going forward.

9. Conclusion & Key Takeaways

This exercise reinforced several important lessons about OS Command Injection and input-filter bypass testing beyond the textbook "single unfiltered separator" example:

- Blocklist filtering keyed on literal substrings or keywords can be systematically defeated by splitting or obfuscating the blocked string, since string matching happens before the target shell reconstructs and executes the final command.
- Isolating variables one at a time — testing the operator independently from the keyword through case modification — is an effective, methodical way to determine exactly which part of an input is being filtered.
- The case-sensitivity of the underlying operating system matters: a payload may successfully evade the application filter yet still fail to execute meaningfully, so understanding what happens after evasion is just as important as achieving the evasion itself.
- Even a single unfiltered character class ($, {}, or backticks) reachable by the shell can be sufficient to defeat an otherwise fairly restrictive keyword blocklist.

Overall, this finding demonstrates a practical, end-to-end OS Command Injection vulnerability: from initial reconnaissance of the ping feature, through systematic filter fuzzing and variable isolation, to a fully confirmed proof-of-concept bypass that achieves arbitrary command execution on the target host. It is documented here as part of ongoing self-study in web application penetration testing.